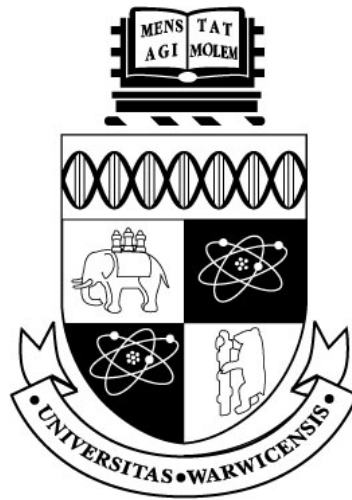




DISPLAY OPTIMISATION AND EXPERIMENTAL EVALUATION OF LARGE VISUAL BEHAVIOUR TREES IN GODOT 4.6

by

[AUTHOR NAME]

A thesis submitted in partial fulfilment
of the requirements for the degree of

MSC GAMES ENGINEERING

UNIVERSITY OF WARWICK

WMG

September 2026

CONTENTS

Contents	ii
List of figures	iv
List of tables	v
Abstract	vi
1 Introduction	1
1.1 Research background	1
1.2 Problem definition and research gap	1
1.3 Aim and research question	2
1.4 Project objectives and success criteria	3
2 Literature review	4
2.1 Behaviour trees as executable hierarchies	4
2.2 Node-link trees, hierarchy and scale	4
2.3 Local overlap avoidance and positional stability	5
2.4 Semantic zoom and focus plus context	6
2.5 Relationship emphasis and edge occlusion	7
3 Research method	8
3.1 Research strategy	8
3.2 Experimental objects	8
3.3 Screen-size profiles	9
3.4 Paired operation of the five features	9
3.5 Data processing and success criteria	10
3.6 Structured developer evaluation	11
3.7 Plugin-function and playable-game validation	11
4 System design and implementation	12
4.1 Overall architecture	12
4.2 Resource model and structural validation	13
4.3 Runtime and node semantics	13
4.4 Editing workflow and fixed baseline display	14
	ii

4.5	Live Debug and blackboard inspection	14
4.6	The current five Display features	14
4.6.1	Smart Drag Reflow	15
4.6.2	Adaptive Zoom Detail	15
4.6.3	Readable Edge Overlay	16
4.6.4	Related Node Focus	16
4.6.5	Fisheye Focus	17
4.7	Experimental recording and reproducibility	17
4.8	Playable five-enemy test game	18
5	Experimental results	19
5.1	Plugin-function and playable-game validation	19
5.2	Formal display data and safety checks	19
5.3	Before-and-after comparison of the five features	21
5.3.1	Smart Drag Reflow	21
5.3.2	Adaptive Zoom Detail	21
5.3.3	Readable Edge Overlay	22
5.3.4	Related Node Focus	22
5.3.5	Fisheye Focus	23
5.4	Results across node scales	23
5.5	Results across screen sizes	25
5.6	Structured developer evaluation	26
6	Discussion	27
6.1	Platform correctness as a prerequisite for display research	27
6.2	Interpretation of the display optimisations	27
6.2.1	Effects across different complex behaviour-tree scales	27
6.2.2	Effects across physical screen sizes	28
6.2.3	Which features are most valuable	28
6.2.4	What the evidence can demonstrate	29
7	Conclusion	30
	References	31

LIST OF FIGURES

4.1	Relationship between the plugin, resources, runtime, debugging bridge and test game	12
4.2	Live Debug on the current version of the real 241-node behaviour tree	15
4.3	Five enemies in the playable scene, each controlled by a different-sized behaviour tree	18
5.1	Smart Drag Reflow off and on: removing card occlusion at the same dragged position	21
5.2	Adaptive Zoom Detail off and on: reducing card area and fields at the same overview zoom	21
5.3	Readable Edge Overlay off and on: the same route appears through a card background while avoiding text	22
5.4	Related Node Focus off and on: keeping the selected nodes and their related branches prominent	23
5.5	Fisheye Focus off and on: restoring target size and dimming distant nodes in an overview	23
5.6	Effects of Adaptive, Related Focus and Fisheye across tree scales	24
5.7	Effects of Adaptive, Related Focus and Fisheye across screen-size profiles . . .	25

LIST OF TABLES

3.1	Resource nodes, canvas cards and Decorators in the five real behaviour trees	8
3.2	Physical-size sources and replayed canvas dimensions for the three profiles	9
3.3	Off-on operation and measures for the five features	10
4.1	Principal semantics of the current runtime nodes	13
5.1	Plugin-function and game-validation results for the current version	19
5.2	Principal result of each feature relative to its off state	20
5.3	Principal display changes across five behaviour-tree scales	24
5.4	Principal display changes under three physical-size profiles	25
5.5	Structured developer evaluation based on consistent questions	26

ABSTRACT

This project developed a visual behaviour-tree plugin for Godot 4.6 and focused mainly on the display problems of large behaviour trees. Conventional node-link editors occupy a large canvas as the tree grows, while dragging, connections and unrelated branches can all interfere with inspection. To address these problems, the current Display menu contains five switchable features: Smart Drag Reflow, Adaptive Zoom Detail, Readable Edge Overlay, Related Node Focus and Fisheye Focus. Search, straight connections, active paths, failure explanations and overview controls are fixed plugin capabilities and are not included in this five-feature comparison.

The experiment uses five genuine game behaviour trees and three physical-screen-size profiles taken from real devices. Each feature is compared in off and on states using the same tree, target and view. The formal experiment produces 225 paired comparisons and also checks tree structure, saved coordinates and sibling execution order. Before-and-after screenshots show the actual visual changes, while a structured developer evaluation explains the types of task for which each feature is more suitable.

The results show that Smart Drag Reflow can remove overlap caused by the controlled drag operation. Adaptive Zoom Detail clearly reduces card area in the overview, while Related Node Focus consistently dims every unrelated node. The revised fisheye restores the focus card to at least approximately 220 px and expands the clear area around the pointer, so its effect is more apparent on small screens and large trees. However, the stronger magnification also introduces more local overlap. Readable Edge Overlay can reveal part of a connection hidden by a card, but it only addresses a local connection problem.

Overall, Adaptive Zoom Detail is the most suitable general display feature. Smart Drag Reflow and Related Node Focus are more appropriate for node editing and structural inspection respectively. Fisheye is suitable for temporarily inspecting a local area from an overview, while Readable Edge Overlay is useful in specific edge-occlusion cases. These automated data describe measurable interface changes, but they cannot replace a usability study with real game developers.

Keywords: behaviour tree; Godot; node-link editor; semantic zoom; focus and context; graph layout; game artificial intelligence

1 INTRODUCTION

1.1 RESEARCH BACKGROUND

This project uses Behaviour Trees (BTs) to control enemies in a game. A behaviour tree divides a decision into composite, condition and action nodes, so high-level strategy can be edited separately from movement, animation and combat code. Putting all logic into one script can still work for a small prototype, but the control flow becomes increasingly difficult to inspect as states and interruption conditions are added. The hierarchical structure of a behaviour tree can reduce this problem, which is one of its main uses in game development (Colledanchise and Ögren, 2018; Iovino et al., 2022).

A node-link tree can show parent-child relationships directly, but the canvas grows as the tree becomes wider and deeper. Behaviour-tree cards also show parameters, descriptions, Decorators, blackboard keys and runtime states, so they normally require more space than an ordinary hierarchy that shows names only. When the canvas contains many cards, a developer needs to zoom, pan and search frequently, and the current execution path can be lost among unrelated branches. This affects node location, condition editing and failure inspection, rather than merely whether the graph looks tidy.

This project therefore first completed a Godot plugin that can save, run and debug genuine NPC decisions, and then used it to test the five display features that remain in the current version. These features address drag occlusion, information density during zooming, cards hiding connections, the relationship scope of selected nodes and local detail in an overview. Each feature can be disabled, which permits direct comparison with the original display and also checks whether disabling it leaves any visual state behind.

1.2 PROBLEM DEFINITION AND RESEARCH GAP

Previous studies approach large graphs from several directions. Tidy tree layout focuses on hierarchy, order and compactness, while layout-adjustment research is more concerned with whether original positions and topology can be preserved after overlap is removed (Reingold and Tilford, 1981; Sugiyama et al., 1981; Misue et al., 1995; Dwyer et al., 2006). Semantic zoom reduces displayed content when zooming out, whereas a fisheye view enlarges the focus while retaining the surrounding structure (Bederson et al., 1996; Furnas, 1986). Interactive emphasis and EdgeLens further address related subgraphs and edge occlusion (Ware and Bobrow, 2005; Wong et al., 2003). Together, these studies show that

large graphs can be improved through layout, information density, focus and edge treatment.

However, these studies do not directly explain how these methods should be combined in a game behaviour-tree editor. A behaviour tree differs from an ordinary directory: sibling order may determine execution priority, leaf nodes call Actor code, and a Decorator can prevent a branch from running. A display feature therefore cannot simply make the graph appear tidier. It must also preserve execution order, support undo, and avoid breaking blackboard and runtime debugging.

Existing work also cannot replace testing within this project because the graphs, tasks and screen conditions used by other studies differ. Reduced card area, changed opacity or target magnification can only show what changed in the interface; they cannot directly prove that a developer understands the tree more quickly. For this reason, the project compares all five features on the same executable platform and records both their effects and their costs.

1.3 AIM AND RESEARCH QUESTION

This dissertation aims to design and evaluate composable display optimisations for large trees on a functionally validated Godot 4.6 visual behaviour-tree platform that can control genuine NPCs. The research question is: across different behaviour-tree node scales and physical screen sizes, can the composable display optimisations proposed in this project improve measurable display and editing-interaction performance for large behaviour trees compared with a conventional baseline display, while preserving behaviour-tree structure and execution semantics?

The principal contributions are:

- a Godot 4.6 behaviour-tree platform that includes editing, resources, a runtime, blackboards, Decorators and Live Debug;
- a Display menu consolidated into five switchable features that address distinct problems, together with a clear boundary between these features and fixed baseline capabilities;
- 225 off-on pairs across five real game behaviour trees and three physical-size profiles;
- a structured developer evaluation that records each feature's benefit, cost, applicable tree scale, applicable screen and recommended state using the same questions; and
- conclusions that distinguish the most stable general feature from features that are most effective in particular conditions, instead of combining measures with different units into a meaningless total score.

1.4 PROJECT OBJECTIVES AND SUCCESS CRITERIA

The project work is divided into platform construction and research evaluation. Platform construction includes serialisable tree, node, Decorator and typed-blackboard resources; success, failure and running semantics for Root, Sequence, Selector, Reactive Selector, Random Selector, Parallel, Repeat, Wait, Action and Condition; a reusable component that assigns a tree and Actor to an NPC and allows leaves to call Actor methods; creation, connection, disconnection, dragging, box selection, multi-node movement, typed editing, validation, saving, loading, undo and redo; and editor-only presentation of the active path, node states, blackboard and failure reasons.

Before the display-optimisation experiment, the basic behaviour-tree plugin must first be shown to work correctly. Each node type must execute as intended; an edited behaviour tree must save and reopen correctly; and undo and redo must not corrupt tree content. The five playable enemies must load the 31-, 61-, 121-, 241- and 364-node trees respectively. The player must have unlimited health, the number of enemies must be finite, and the game must end after all enemies have been defeated.

The display study has three kinds of success criterion. First, a feature must produce its claimed direct change: for example, Smart Drag must reduce overlap caused by dragging, Adaptive must reduce overview card area, and Related Focus must dim unrelated nodes. Second, a feature must not change tree topology, Decorator ownership or sibling execution order; automatically moved neighbours may use only temporary display offsets. Third, disabling a feature must not leave residual size, opacity, frame or temporary layout state. A script error, crash, memory leak, illegal access or abnormal exit causes the test to fail even if the remaining assertions pass.

LITERATURE REVIEW

2

2.1 BEHAVIOUR TREES AS EXECUTABLE HIERARCHIES

A behaviour tree repeatedly evaluates a rooted hierarchy and returns success, failure or running. A Sequence normally stops at the first child that fails or is still running, while a Selector stops at the first child that succeeds or is still running. Marzinotto et al. (2014) proposed a unified behaviour-tree framework, while Colledanchise and Ögren (2017, 2018) later analysed this form of composition from the perspectives of hybrid control and modularity. These studies show that the basic nodes appear simple, but Tick, running, interruption and subtree composition require explicit rules. The review by Iovino et al. (2022) also notes that precise semantics can still differ between fields and tools.

Behaviour trees have been used in genuine game tasks. Flórez-Puga et al. (2009) added queries and reuse of existing behaviours to game behaviour trees, while Lim et al. (2010) evolved a game agent for the commercial real-time strategy game DEFCON. Although their aims differ, both studies show that a behaviour tree can organise non-trivial game decisions containing multiple conditions and actions rather than only small demonstrations.

This also explains why the runtime is validated before the display experiment in this project. A Sequence can remember its current execution position, whereas a Reactive Selector needs to recheck high-priority conditions. A Random Selector should retain one choice during a running episode, while Repeat and Wait must clear their memory when restarted. A blackboard shares state between perception, conditions and actions, while a Decorator limits or changes branch execution without modifying the child node itself. The editor cannot treat node order and Decorators as purely visual information because they affect execution results. However, these basic capabilities are correctness prerequisites for the experimental platform, rather than separate research objects whose usefulness needs to be proved in this dissertation.

2.2 NODE-LINK TREES, HIERARCHY AND SCALE

Node-link diagrams are suitable for behaviour trees because connections can show parent-child relationships directly. The tidy-tree layout proposed by Reingold and Tilford (1981) emphasises centred parents, spacing within a level and separated subtrees, while Sugiyama et al. (1981) further discuss within-layer ordering and node positions. Taken together, these methods are useful for generating an orderly tree automatically, but they do not com-

pletely solve interactive editing because users still want to drag nodes freely.

Whether a layout is better cannot be judged only from whether it looks tidy. Purchase (1997) found that edge crossings had a strong effect among the tested graph aesthetics. Ware et al. (2002) further show that path continuity, crossings and branches affect path finding in node-link diagrams. Results from Purchase, Carrington and Allder (2002) indicate that only some aesthetic measures consistently affect task performance, and that the semantic domain of the graph can also change the outcome. This dissertation therefore records direct measures such as overlap, area and structure, but does not describe these values as a direct improvement in human understanding.

As scale increases, both nodes and connections become more difficult to inspect. Ghoniem et al. (2005) compare node-link and matrix representations and show that results are affected by size, density and task together. SpaceTree adds dynamic layout and selective display to a node-link tree, also showing that representation and interaction need to be evaluated together (Plaisant et al., 2002). For behaviour trees, the node-link representation is still the clearest way to show parent-child direction and sibling order. This project therefore retains it and controls information density only during zooming and selection.

Recent visual workflow systems show that large node canvases are no longer limited to conventional programming tools. InstructPipe lets users generate and continue editing machine-learning workflows in a node graph (Zhou et al., 2025). Although this system does not study behaviour trees or answer the display questions in this project directly, it shows that node positioning, connection tracing and navigation through complex workflows remain practical issues in recent development tools.

Recent research also warns that general layout rules cannot be separated from domain semantics. Helmke et al. (2024) found that domain experts sometimes prioritise preservation of a technical structure instead of always reducing crossings or edge length. Behaviour trees similarly have parent-child direction and sibling execution order, so automatic layout cannot optimise only for minimum area. The systematic review by Di Bartolomeo et al. (2024) also shows that algorithms, datasets, scales and metrics need to be reported together when graph layouts are evaluated. This supports the use of genuine behaviour trees, several scales and several direct measures in this dissertation, rather than a single total score with no clear meaning.

2.3 LOCAL OVERLAP AVOIDANCE AND POSITIONAL STABILITY

Interactive editing differs from a one-off automatic layout. If the system rearranges the whole tree after the user drags one node, overlap may disappear but the memory of the original positions may also be disrupted. Misue et al. (1995) described the problems of layout adjustment and the mental map, while Dwyer et al. (2006) proposed that nodes should be separated with their new positions remaining as close as possible to their originals. Later work also showed that existing topology needs to be preserved when constraints

are introduced (Dwyer et al., 2009).

Behaviour-tree cards have real width, height, text and labels, so they cannot be treated simply as points with no area. Gansner and Hu (2010) emphasise preservation of proximity and the original graph shape during overlap removal. This principle suggests the direction of Smart Drag: the system should handle new overlap, but it should not use this as a reason to take control of the complete canvas.

This project does not directly reproduce one complete published algorithm. Instead, it limits activation distance, update frequency, propagation depth and the maximum affected region, and then uses parent-child and sibling relationships to decide which nodes should avoid each other together. The system preserves only parent-above-child relationships that were already valid before the drag. If the user originally chose a free layout, the plugin does not force it into a standard tree arrangement.

Experimental work on dynamic graph drawing shows that the effects of position preservation can depend on the task (Archambault and Purchase, 2013). This project therefore treats unchanged structure and minimal movement of unrelated nodes as engineering requirements rather than describing them directly as a usability improvement. Whether the tree is genuinely easier to understand still requires a user study.

2.4 SEMANTIC ZOOM AND FOCUS PLUS CONTEXT

Semantic zoom does more than make a view smaller; it also changes the displayed content. Pad++ shows that a representation can change with the zoom level (Bederson et al., 1996), while Summers et al. (2003) compared different semantic-zoom methods in program visualisation. These studies show that hiding secondary fields while zoomed out and restoring detail after zooming in is a viable approach. However, their programs and tasks differ from behaviour-tree editing, so their reported speed or accuracy cannot be transferred directly to this project.

Focus-and-context methods address a different problem. Furnas (1986) uses degree of interest to distinguish the focus from surrounding content, Sarkar and Brown (1994) apply geometric fisheye transformations to graphs, and Wang et al. (2019) propose a structure-aware fisheye for reducing distortion in large graphs. These methods are more complex than the card magnification currently implemented in this project, but they show that fish-eye should not be evaluated only by its magnification factor; structural change and local occlusion also need to be checked.

The review by Cockburn et al. (2009) notes that overview, zoom and focus plus context each have costs and that no method suits every task. Büring et al. (2006) also found no necessary completion-time advantage for fisheye in a small-screen scatterplot study, although more participants preferred it. In other words, fisheye may support local inspection, but it still needs to be tested separately in the current behaviour-tree editor.

Physical screen size changes the actual size of cards in an overview. Jakobsen and Hornbæk (2013) show that display size, information space and scale can affect interaction to-

gether. However, the tasks in that study are not behaviour-tree editing, so its results only show that screen size is worth including as an experimental condition; they cannot supply an effect size for this project.

2.5 RELATIONSHIP EMPHASIS AND EDGE OCCLUSION

A large graph does not always need to hide nodes first. Ware and Bobrow (2005) use interactive emphasis to help users query node–link diagrams, while van Ham and Perer (2009) retain context around a target of interest through degree-of-interest exploration. For a behaviour tree, this approach can preserve original spatial positions while reducing the visual intensity of unrelated branches. Related Node Focus therefore uses frames and opacity to distinguish selected nodes, ancestors, descendants, siblings and unrelated branches.

Connections can also be hidden by nodes or other edges. EdgeLens by Wong et al. (2003) bends connections near a focus to make space in a congested region, while the hierarchical edge bundles proposed by Holten (2006) organise many connections along the hierarchy. The experiment by McGee and Dingliana (2012) shows that bundling may help inspection of overall clusters but can reduce the speed and accuracy of tracing an individual path. Reducing visual clutter therefore does not mean that every connection task becomes easier.

This project does not implement EdgeLens or edge bundling and does not change connection routes. Readable Edge Overlay only allows a line to appear through a semi-transparent card background and uses text outlines to protect glyphs. Its method is simpler and its scope is more limited. Taken together, previous work has considered layout, zoom, focus, highlighting, connections and recent visual workflows separately, but it has not compared this combination in a genuine Godot behaviour-tree editor. The research gap in this project is not the proposal of a completely new graph algorithm. Instead, it turns five approaches into practical features that can be enabled, disabled and reset, and then uses real game trees, different scales and different screen sizes to examine the change and cost produced by each feature.

RESEARCH METHOD

3

3.1 RESEARCH STRATEGY

This study first confirms that the Godot 4.6 behaviour-tree plugin can be edited and run correctly, and then tests its five large-tree display features. The focus is not whether behaviour trees are suitable for making games, but what these features change under different tree scales and screen sizes.

The evaluation has two parts. The first disables and enables each feature and records area, overlap, opacity, target width and structural safety. In the second, the project developer uses the same questions to assess practical help, the main problem, applicable environment and recommended state. The second part is only a structured personal evaluation. It is not a participant study and cannot represent the average opinion of other developers.

3.2 EXPERIMENTAL OBJECTS

The experiment uses the behaviour trees actually loaded by the five enemies in the current playable game. Resource nodes include Decorators attached to other nodes, whereas Decorators are not displayed as separate canvas cards. Resource-node counts and canvas-card counts must therefore be reported separately.

TABLE 3.1: Resource nodes, canvas cards and Decorators in the five real behaviour trees

Resource nodes	Canvas cards	Decorators	Resource	Game Actor
31	30	1	arena_scout_31.tres	Scout
61	56	5	arena_skirmisher_61.tres	Skirmisher
121	104	17	arena_hunter_121.tres	Hunter
241	202	39	arena_tactician_241.tres	Tactician
364	301	63	arena_commander_364.tres	Commander

All five trees contain executable composite nodes, conditions, actions and Decorators rather than empty structures created for screenshots. The larger trees cover directional melee combat, ranged projectiles, obstacle jumping, ladder climbing, pursuit, last-known-position search, retreat, healing, patrol and idle behaviour. Branch shape and field content

also change as scale increases, so the experiment can compare behaviour under genuine complexity, but every difference between adjacent scales cannot be interpreted as a pure causal effect of node count.

3.3 SCREEN-SIZE PROFILES

The screen conditions are derived from EDID records of the physical width and height of three devices collected on 23 August 2026. A consistent density of 35 logical units per centimetre is used to generate each SubViewport. Only the laptop screen is currently enabled, so the formal experiment replays the three physical-size profiles on the same GPU. It does not claim that the window was rerun separately on the three physical displays.

TABLE 3.2: Physical-size sources and replayed canvas dimensions for the three profiles

Profile	Physical height	width	×	Diagonal	Physical area	SubViewport Usable area	GraphEdit
Laptop	34×22 cm			15.94 in	748 cm ²	1190×770	858×642
Medium	60×33 cm			26.96 in	1980 cm ²	2100×1155	1768×1027
Large	70×39 cm			31.55 in	2730 cm ²	2450×1365	2118×1237

Resolution, refresh rate and Windows scaling are retained only as device-audit information and are not used to explain effects. Whether a node lies inside the viewport is calculated from the usable GraphEdit area rather than the complete SubViewport, which also contains the toolbar and Inspector.

3.4 PAIRED OPERATION OF THE FIVE FEATURES

Three fixed targets are selected in advance from the left, centre and right of each tree to cover different branches. They are not random samples. Before each record, the script copies the tree resource, rebuilds the canvas and resets feature states, so temporary state from one record cannot enter the next. Each pair records the off state first and the on state second.

For Smart, Adaptive, Overlay and Related, the off state disables all five experimental features. Fisheye is the only exception: because it is intended for use in an overview, both its off and on states retain Adaptive and differ only in whether fisheye is additionally applied. This design measures the incremental effect of fisheye relative to an adaptive overview.

TABLE 3.3: Off-on operation and measures for the five features

Feature			Off state and operation	On state and principal measures
Smart flow	Drag	Re-flow	Drag a fixed leaf into the region of a fixed target card and record the deliberately induced occlusion	Repeat the same drag path; compare overlap area, number of automatically moved cards, maximum displacement and movement of distant branches
Adaptive Detail	Zoom		Continue to show full cards at a fixed overview zoom	Switch to medium/low detail according to zoom; compare card area, field count, cards fully within the viewport and hierarchy violations
Readable Overlay	Edge		Fix a real parent-child connection that passes through a non-endpoint card, with the card kept opaque	Multiply the background by 0.72; compare the segment crossing the card, weighted visible length, text-mask styling and route stability
Related Node Focus	Node	Fo-	Single-click the fixed node without applying relationship emphasis; the third task uses box selection to create two selected nodes	Take the union of ancestors, descendants and sibling relationships for all selected nodes; check white, yellow and green frames, dimming of unrelated nodes and connections, and unchanged geometry
Fisheye Focus			Retain the same Adaptive overview without applying fisheye	Hold the pointer near the fixed target; compare target width, restored fields, peripheral shrinking and dimming, and new occlusion

The total number of state records is $5 \text{ features} \times 3 \text{ screen profiles} \times 5 \text{ trees} \times 3 \text{ tasks} \times 2 \text{ states} = 450$. These records form 225 one-to-one comparisons, with 45 pairs for each feature. Each state waits for two rendered frames before recording. The experiment uses Godot 4.6 stable, the OpenGL Compatibility renderer and an NVIDIA GeForce RTX 5070 Laptop GPU.

3.5 DATA PROCESSING AND SUCCESS CRITERIA

The analysis aligns off and on states using `pair_id` and gives equal weight to each screen-tree-task condition. Because the tasks and interface states are deterministic, the three tasks are fixed branch cases rather than samples drawn randomly from a population. The dissertation therefore reports paired means, medians, ranges and the number of conditions that satisfy a feature contract. It does not calculate p-values or confidence intervals without a clearly defined population.

All pairs share the following checks: resource-node count, parent-child relationships, Decorator ownership, left-to-right sibling execution order and coordinates of non-dragged

resource nodes remain unchanged; disabling a feature leaves no residual size, opacity, frame or temporary offset; and logs contain no script error, crash, leak or non-zero exit. The formal Smart fixture cancels the drag and restores the source node after recording, allowing saved coordinates to be compared exactly with their pre-experiment state. In normal use, a node actively dragged by the user saves its new coordinate and enters Undo/Redo, while automatically avoided neighbours remain absent from the resource.

Each feature also has its own success criteria. Smart must reduce overlap caused by dragging and limit the range of automatic movement. Adaptive must reduce card area or fields without creating new parent–child vertical-order errors. Overlay must reveal part of an occluded segment while preserving text and its connection route. Related must handle every selected node correctly and dim every unrelated node. Fisheye must select the correct target and clearly magnify it; whether it reaches the current design target of approximately 220 px is recorded separately. Fisheye is designed to tolerate peripheral occlusion and is therefore not required to keep the whole graph overlap-free, but every new overlap must be reported.

3.6 STRUCTURED DEVELOPER EVALUATION

The structured developer evaluation uses the same trees, targets and before-and-after screenshots. Each feature answers five questions: which task it helps, its most apparent problem, the scale and screen on which it is more useful, and whether it should be enabled by default or on demand. Each judgement must be traceable to a table or screenshot rather than personal preference alone.

This evaluation can explain practical trade-offs that automated measures do not express, such as whether neighbour movement is acceptable or whether fisheye occlusion interferes with inspection. However, it still represents only the developer of this project. Task times, error rates and subjective ratings from real participants remain future work.

3.7 PLUGIN-FUNCTION AND PLAYABLE-GAME VALIDATION

The display experiment depends on a functionally correct platform. Validation is divided into resource and runtime tests, editor-interface tests, basic-game tests, complex-arena tests and the five-enemy completion flow. It covers node execution, Decorators, Actor method calls, creation and connection, save and reload, undo and redo, multi-selection dragging, display-feature reset, permanent enemy defeat and game completion.

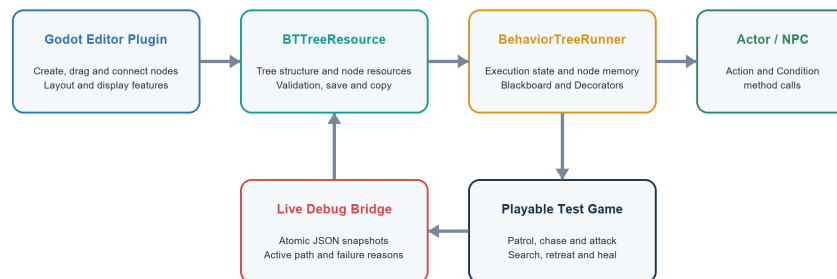
Playable validation does not address an additional research question about whether behaviour trees help game development. It only confirms that the five experimental trees are loaded by game Actors, that complex behaviour executes, that the player has unlimited health, that the five finite enemies can be defeated one by one, and that a completion state is eventually reached. The dissertation therefore evaluates more than a set of static graphs detached from a runtime.

SYSTEM DESIGN AND IMPLEMENTATION

4.1 OVERALL ARCHITECTURE

The system consists of five parts: an editor plugin, a resource model, a runtime, a debugging bridge and a test game. The editor plugin runs within the Godot editor and presents a behaviour-tree resource as an interactive GraphEdit canvas. The resource model stores nodes, parent-child relationships, Decorators, the blackboard Schema and node coordinates. The runtime reads the same resource and passes tick results to the game Actor. The debugging bridge returns current state, failure reasons and blackboard values to the editor. The test game supplies an executable environment only and does not overlay a behaviour-tree debugging interface on the game view.

Plugin Architecture and Data Flow



The editor visualises the resource; the runner executes it; Live Debug returns runtime state to the editor.

FIGURE 4.1: Relationship between the plugin, resources, runtime, debugging bridge and test game

This separation allows the same tree to be edited, saved, tested automatically and run in the game. Display features change only temporary card size, opacity, frame or canvas offset; they do not rewrite node types or connections. A node that the user deliberately drags and releases is the exception: its new coordinate is saved normally and can be undone or redone. Other cards moved automatically for avoidance are not written back to the resource.

4.2 RESOURCE MODEL AND STRUCTURAL VALIDATION

A behaviour-tree resource stores the root and the node collection. Every node has a stable identity, type, editor coordinate, parameters and child references. A Decorator is attached to its owning node rather than represented as an independent canvas card. The blackboard Schema defines key names, types and default values. Actions and Conditions use typed controls to select methods, keys, comparison operators and values, so users do not need to write JSON manually.

Pre-save validation rejects multiple roots, cycles, disconnected invalid references, incorrect child counts, invalid Decorator ownership and blackboard keys that do not conform to the Schema. Cards and connections are reconstructed after loading. Sibling nodes are ordered from left to right by horizontal coordinate, and this order affects the execution priority of Sequence, Selector and other ordered composite nodes. Display reflow must therefore not silently change resource x-coordinates or sibling execution order.

4.3 RUNTIME AND NODE SEMANTICS

Every runtime tick returns SUCCESS, FAILURE or RUNNING. Root passes the result from its only child. Sequence runs children from left to right and stops at a child that fails or remains running. Selector stops at a child that succeeds or remains running. Reactive Selector rechecks high-priority branches on every tick. Random Selector retains the selected child for one running episode. Parallel counts success and failure according to its configuration. Repeat controls a finite or infinite repetition. Wait records elapsed time. Action calls an Actor method. Condition checks an Actor method or blackboard value.

TABLE 4.1: Principal semantics of the current runtime nodes

Node	Principal responsibility	Runtime memory that must be retained
Root	Enter the whole tree	None
Sequence / Selector	Combine results in sibling order	Current running child
Reactive Selector	Recheck from highest priority each time	Reset the previous branch when pre-empted
Random Selector	Select a child randomly	Current selection and random state
Parallel	Evaluate several children concurrently	State of each child
Repeat / Wait	Repeat or wait	Count or elapsed time
Action / Condition	Call the Actor or read state	Depends on the task

Before and after its owning node executes, a Decorator applies a blackboard condition, cooldown, time limit, result inversion, forced result or repetition rule. When a branch is interrupted, running nodes and Decorators are reset so that an incorrect state is not inherited on the next entry. The runtime uses the tree resource but does not depend on the editor being open, so disabling every display feature does not change game results.

4.4 EDITING WORKFLOW AND FIXED BASELINE DISPLAY

The user right-clicks empty canvas space to create a node, creates parent–child relationships through ports, and edits parameters through typed fields in the Inspector. Nodes can be deleted, disconnected, box-selected and dragged as a group. Clicking a node clears the previous single selection and selects the current node; only box selection or a modified selection operation creates a multi-selection. Holding the pointer on empty space pans the view. Every structural edit, parameter edit and user drag is entered into Undo/Redo and can be saved to a .tres resource and loaded again.

The editor displays parent–child relationships using fixed straight connections and applies consistent colours to ports, selection state and runtime state. Search, Minimap, active-path highlighting, inactive-branch dimming, Decorator condition badges and failure explanations are fixed baseline capabilities. They do not have research switches in the current Display menu and are not part of the five off–on comparisons. This distinction prevents baseline navigation and runtime debugging from being confused with the five switchable optimisations investigated here.

4.5 LIVE DEBUG AND BLACKBOARD INSPECTION

While the game is running, the debugging bridge records active nodes, node return values, failure reasons and a blackboard snapshot for the current Actor. The editor presents the active path on the original behaviour-tree canvas and displays the current value and type of each blackboard key in the side panel. A failure explanation is placed near the node at which the failure occurs, for example when a condition is false, a method is missing or a Decorator blocks a branch.

The debugging interface does not add an overlay to the running game and does not require the Actor to know about editor UI. Its purpose is to demonstrate that the experimental trees are executable and to let a developer inspect where execution currently is and why a branch was not entered. It is not an independent condition in the five-feature display experiment.

4.6 THE CURRENT FIVE DISPLAY FEATURES

The Display menu now contains only the five switchable features included in this research. Factory settings for a fresh installation enable Smart Drag, Adaptive, Related Focus and

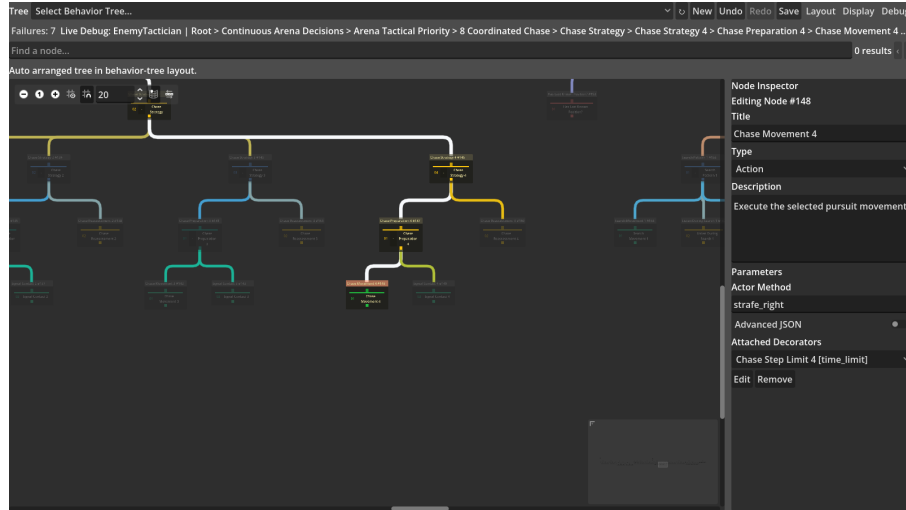


FIGURE 4.2: Live Debug on the current version of the real 241-node behaviour tree

Fisheye and disable Edge Overlay; an existing project restores the user's previously saved choices. The formal experiment does not depend on factory settings, because it explicitly disables or enables the tested feature for each pair.

4.6.1 SMART DRAG REFLOW

Smart Drag Reflow addresses new occlusion caused by dragging one node onto another. It activates only after cumulative movement reaches 10 screen pixels and then updates after each further 8 screen pixels. A click or a very small correction does not trigger reflow. Temporary avoidance is updated during the drag, allowing the user to see other cards move aside before the mouse button is released.

The algorithm first finds the local structure colliding with the dragged card and then moves related parents, siblings and children within two to four consecutive levels as small groups. It processes at most 24 cards in one operation and limits collision-propagation passes. This reduces the risk of the whole tree suddenly being forced into regular rows. It preserves a parent-above-child relationship only if that relationship was valid before the drag; if the user deliberately placed a group in an inverted arrangement, the system does not force it into a standard tree layout.

Smart reflow is used for a single-node drag. A box-selected multi-node drag is treated as one rigid group and does not invoke the solver for each node, preventing the relative positions within the group from changing. On release, only the new coordinate of the user-dragged node is saved; displacement of automatically avoided cards remains display-layer data. Disabling the feature or rebuilding the canvas clears these temporary offsets.

4.6.2 ADAPTIVE ZOOM DETAIL

Adaptive Zoom Detail links card size and field count to the GraphEdit zoom. Below 0.62 zoom, it uses a compact card of approximately 188×88 and retains only the low-detail infor-

mation needed to identify the structure. Between 0.62 and 0.88, it restores normal size and a medium set of fields. At 0.88 and above, it presents an approximately 250×150 normal card with all fields.

After card size changes, the system checks the entire canvas for collisions and uses temporary offsets to preserve the existing relative structure where possible. This is not a permanent replacement of the tree layout and is not Smart Drag’s local 24-card solver. Fields and normal card size return when the view is enlarged, and disabling the feature also clears adaptive sizes and offsets.

This feature exchanges information density for overview space. Its purpose is not to keep all text readable at the smallest zoom, but to let the user first see the tree shape and branch extent and then enlarge the view to inspect parameters when necessary. The experiment records changes in area, field count, cards completely inside the viewport and hierarchical relationships separately.

4.6.3 READABLE EDGE OVERLAY

Readable Edge Overlay addresses a local case in which a connection is hidden by a non-endpoint card. When enabled, supported card backgrounds multiply their original opacity by 0.72 so that a straight line behind the card can become partially visible. Text, icons and controls use fixed foreground colours with a 2 px dark outline, preventing the revealed line from passing directly through glyphs. The route, endpoints and execution order of the connection remain unchanged.

The feature applies transparency only to card backgrounds using `StyleBoxFlat`. Disabling it restores the original background, text style and modulation colour. It is not Edge-Lens: the system does not curve, bundle or reroute connections. It also does not reduce the number of nodes, card area or whole-graph edge density, so its expected effect is more local than that of the other four features.

4.6.4 RELATED NODE FOCUS

Related Node Focus calculates structural relationships after the user selects one or more nodes. The selected nodes use white frames; all their ancestors and descendants use yellow frames; other nodes under the same parent use green frames; and every unrelated node and connection becomes much dimmer. The feature does not move nodes or change card sizes, so the original tree positions remain visible.

An ordinary click selects only the current node and does not keep accumulating nodes that were clicked previously. When the user needs to select several nodes, a selection box can be dragged from empty canvas space before the nodes are moved or focused together. Multi-selection focus takes the union of the ancestors, descendants and siblings of every selected node, so each selected node contributes to the result rather than only the last one. If a Decorator is selected, the system first maps it to its owning visible card. Clearing the selection restores every frame and opacity value.

Related Focus and the runtime active path are separate layers. The former answers which nodes are structurally related to the selection; the latter answers what the Actor is currently executing. They can coexist, but the formal experiment compares only the on/off change in structural relationship emphasis.

4.6.5 FISHEYE FOCUS

Fisheye Focus is mainly intended for an overview that has already been zoomed out. It does not magnify only the single card directly under the pointer. Instead, it continuously changes card size according to the distance between each node and the pointer centre. The node closest to the centre is enlarged the most, nearby nodes receive a smaller increase, and nodes outside the main range shrink and fade. The aim is to make one local area readable without first leaving the overview.

The revised focus radius is 150 px, the main transition range is 320 px, and the peripheral fading range has been expanded to 600 px. The system calculates the magnification dynamically from the current zoom and restores the focus card to at least approximately 220 px. The minimum focus magnification is 1.20 $\times$ and the maximum is 8.8 $\times$. Distant cards shrink to 0.68 $\times$ and their opacity falls as low as 0.12. The smaller the current zoom, the stronger the contrast between the focus and the periphery.

Only the eight nearest cards can participate in limited local avoidance, so other nodes do not push the whole canvas when the focus becomes larger. Distances are calculated from stable reference positions, and both the focus position and card dimensions are quantised into buckets. Small pointer tremors therefore do not cause reflow on every frame. Fisheye pauses briefly during mouse-wheel zoom and resumes afterwards.

This design prioritises clarity of the focus and nearby nodes rather than a completely overlap-free tree. Stronger magnification can cover nearby cards and can temporarily disrupt the visual parent-above-child relationship. The experiment therefore reports new overlap and hierarchy occlusion separately from target magnification.

4.7 EXPERIMENTAL RECORDING AND REPRODUCIBILITY

The five-feature experiment uses a Godot script to construct genuine editor views directly. It copies the tree resource, sets the screen profile and task target, switches one feature, waits for rendering and records a CSV row. Each record contains the experimental commit, tree-resource hash, screen profile, target, switch state, feature measures and safety signatures. The analysis script reads only the raw CSV and generates a paired table, summaries by tree and screen, a structured evaluation table and before-and-after comparison figures.

The formal data directory stores the raw observations, paired results, execution-environment manifest, SHA-256 hashes of the source resources, ten off-on evidence images and every derived table. The five comparison figures in the dissertation use this same formal evidence set, so the screenshots, tables and raw records can all be traced to one directory.

4.8 PLAYABLE FIVE-ENEMY TEST GAME

The test game assigns the five differently sized behaviour trees to Scout, Skirmisher, Hunter, Tactician and Commander. The number of enemies is finite, the player has unlimited health, and the objective is to defeat each enemy using movement, jumping and attacks. The scene contains ranged projectiles, directional melee attacks, obstacles, jumping routes, ladders, healing items and hazardous areas. Enemies can patrol, detect the player, pursue, search the last known position, traverse obstacles, climb, retreat and recover.



FIGURE 4.3: Five enemies in the playable scene, each controlled by a different-sized behaviour tree

The smaller trees use fewer branches to provide basic behaviour, while larger trees handle more combat and environmental reactions. The game reaches a completion state after all five enemies have been permanently defeated. This demonstrates that the behaviour-tree platform is not a static graph viewer, but game completion is not treated as evidence of display-optimisation effectiveness.

EXPERIMENTAL RESULTS

5.1 PLUGIN-FUNCTION AND PLAYABLE-GAME VALIDATION

Five groups of automated tests were rerun on the current version. They cover the execution system, editor operations, the basic game scene, complex enemy behaviour and the complete five-enemy flow. Every test completed normally, and there were no program errors or crashes during the run. The exact numbers for each group are shown in Table 5.1.

TABLE 5.1: Plugin-function and game-validation results for the current version

Validation suite	Passed/total	Principal scope
Execution-system tests	154/154	Node execution, Decorators, blackboards, save and reload
Editor-function tests	337/337	Creating and editing nodes, selection, dragging, display optimisation and reset
Basic game tests	40/40	Game actors can receive behaviour-tree actions and the basic scene works
Complex game-behaviour tests	34/34	Enemy combat, movement, recovery, obstacles and ladders
Complete game-flow tests	215/215	Five tree scales load, enemies can be defeated and the game can finish
Total	780/780	All automated checks in this run

Each of the five behaviour trees controls its corresponding enemy, and the game can be completed. The following display results are therefore based on real resources that load and execute correctly.

5.2 FORMAL DISPLAY DATA AND SAFETY CHECKS

The formal experiment produced 225 before-and-after comparisons. In other words, the same display optimisation was tested repeatedly under different node counts, screen sizes and target nodes. Each pair changed only one display optimisation switch, while the other conditions stayed the same. This makes it possible to compare the change caused by that one function in that situation.

Every pair passed the feature and structure checks. The behaviour-tree connections, execution order and saved positions did not change. The Smart test restores the dragged node after recording, so coordinate equality here means that automatic avoidance did not write to the resource. In normal editing, a position deliberately created by the user is still saved.

TABLE 5.2: Principal result of each feature relative to its off state

Display optimisation	Before activation	After activation	Main change	Point to watch
Smart Drag Reflow	Dragging creates overlap, with a mean overlap area of 93,834.90 px ²	Overlap area is reduced to 0 in 45/45 cases	Removes the deliberately created node occlusion	Moves a mean of 2.40 other cards; median total movement is 208.54 px
Adaptive Zoom Detail	Full cards and fields	Mean total card-area reduction of 45.14%	Mean field reduction of 43.61%	Fields deliberately hidden at low zoom
Readable Edge Overlay	Card background is fully opaque, so a line through a card can be hidden	Background opacity is reduced to 0.72 while text remains opaque	Median controlled occluded segment is 199.50 px and the route is unchanged	Helps local line occlusion, but does not reduce node count
Related Node Focus	Unrelated nodes keep normal brightness	All unrelated nodes are dimmed	Mean reduction of 58.51% in fully bright viewport candidates	Not suitable for comparing unrelated branches at the same time
Fisheye Focus	Mean target width is 119.43 px in overview	Mean target width is 224.55 px	Local target becomes much larger and restores more fields	New local overlap in 25/45 cases and new visual hierarchy occlusion in 28/45

Each row in Table 5.2 measures a different task, so the values should not be added into a single total score. The following sections explain each display optimisation separately. All before-and-after figures use the 241-node behaviour tree, the approximately 27-inch screen profile and the same test target. The screenshots include only the behaviour-tree editing canvas, not the Inspector panel.

5.3 BEFORE-AND-AFTER COMPARISON OF THE FIVE FEATURES

5.3.1 SMART DRAG REFLOW

Smart Drag Reflow handles new occlusion caused when the user drags a node. In the experiment, the script drags the same leaf node close to another node and deliberately creates overlap. With the function disabled, the overlap remains. With the function enabled, nearby cards move away without changing the saved behaviour-tree resource.

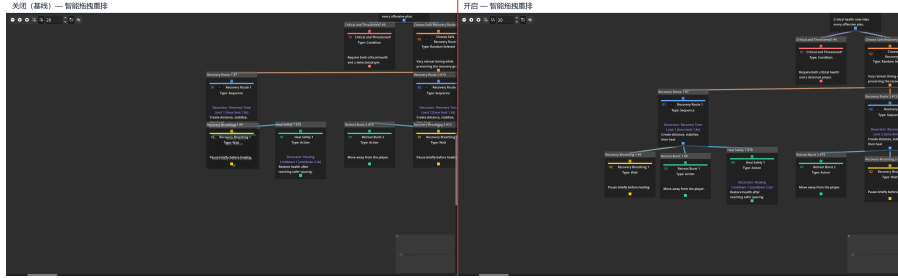


FIGURE 5.1: Smart Drag Reflow off and on: removing card occlusion at the same dragged position

Figure 5.1 shows the difference at the same dragged position. After activation, the overlap area is reduced to zero in all 45 cases. The function does not force the whole tree into a fixed layout; it moves a small part of the structure related to the current drag. On average, 2.40 other cards move; the median is one card and the maximum is seven cards. The median total movement of all other cards is 208.54 px, and the maximum is 906.37 px. This shows that the function is effective for drag occlusion, while the user still needs to accept that a small number of surrounding cards can move.

5.3.2 ADAPTIVE ZOOM DETAIL

Adaptive Zoom Detail addresses the overview problem. When a behaviour tree is zoomed out, full cards occupy a large amount of space. If everything is simply scaled down, text becomes difficult to read. This function hides secondary fields and uses more compact cards at low zoom.

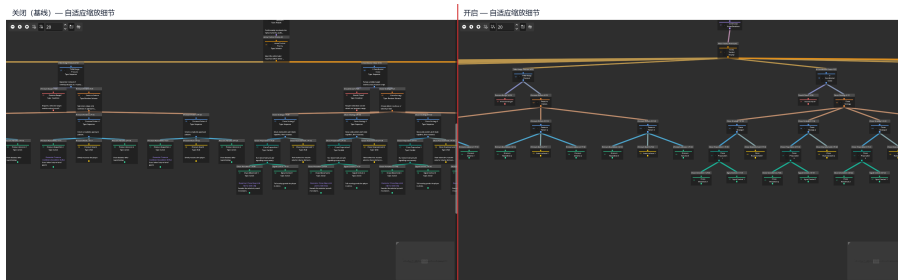


FIGURE 5.2: Adaptive Zoom Detail off and on: reducing card area and fields at the same overview zoom

In Figure 5.2, the on state keeps node identity, type and main structural information, but it no longer shows every parameter. Across the experiment, total card area falls by a mean of 45.14%, and the number of fields falls by a mean of 43.61%. No new case appears where a parent is visually below its child. This makes it suitable as the basic overview feature for large trees: the user first sees the structure and branch range, then zooms in to read exact parameters.

5.3.3 READABLE EDGE OVERLAY

Readable Edge Overlay addresses the local problem where a connection is hidden by a card. The experiment has two parts. First, it scans the real viewport and records how many visible connections pass through non-endpoint cards. Second, it uses the same real parent-child connection and a non-endpoint card to create a controlled occlusion case, and then compares the off and on states.

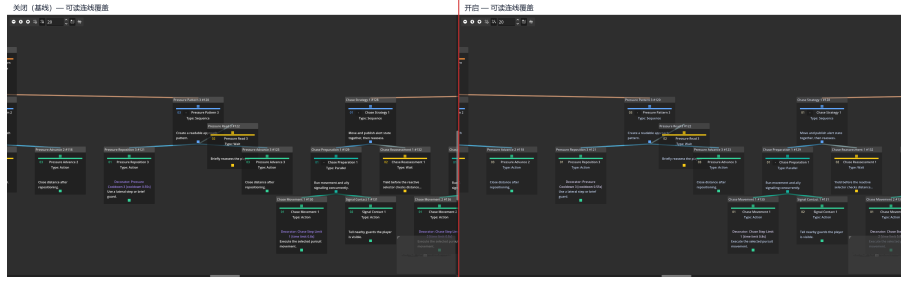


FIGURE 5.3: Readable Edge Overlay off and on: the same route appears through a card background while avoiding text

The natural viewport scan finds a mean of 3.62 visible connections occluded by non-endpoint cards in each case. In the controlled case, the median hidden segment length is 199.50 px. After activation, the card background becomes translucent, but text and icons remain opaque, and the route of the connection does not change. This means the feature can reveal part of a segment that was covered by a card background. Its effect is local: it does not reduce the number of nodes, and it does not make the whole tree less dense.

5.3.4 RELATED NODE FOCUS

Related Node Focus is used to inspect the structural range of the selected node. After activation, selected nodes use a white border, ancestors and descendants use a yellow border, siblings use a green border, and unrelated nodes and edges are dimmed. Because the current editor supports box selection, the experiment also includes one case where two nodes are selected together.

The results show that all unrelated nodes are dimmed correctly, and the number of fully bright candidates in the viewport falls by a mean of 58.51%. Node positions, card area and edge routes remain unchanged. The user can still see where the complete tree is,

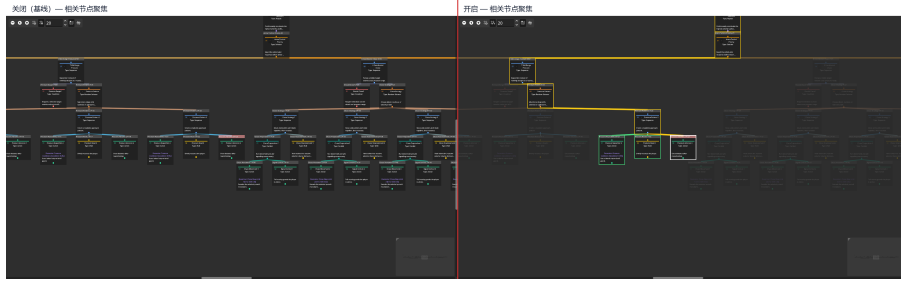


FIGURE 5.4: Related Node Focus off and on: keeping the selected nodes and their related branches prominent

but attention is more easily directed toward the parents, children and siblings related to the current selection.

5.3.5 FISHEYE FOCUS

Fisheye Focus is mainly used in a zoomed-out overview. The off state keeps Adaptive Zoom Detail enabled. The on state adds fisheye magnification around the pointer and makes far nodes smaller and dimmer. This tests whether the user can temporarily read one local area without leaving the overview.



FIGURE 5.5: Fisheye Focus off and on: restoring target size and dimming distant nodes in an overview

After activation, mean target width rises from 119.43 px to 224.55 px, and more fields are restored. This change is visually clear, especially when a large tree has been strongly reduced. However, fisheye also has the clearest cost: 25/45 cases introduce new local overlap, and 28/45 introduce new visual hierarchy occlusion. Tree structure and execution order remain unchanged, but the view no longer guarantees global tidiness. Fisheye is therefore more useful for brief local inspection than as a continuously enabled main layout.

5.4 RESULTS ACROSS NODE SCALES

Table 5.3 summarises the results across different node scales. To keep the table readable, each display optimisation is represented by the metric that best matches its purpose. Adaptive uses card-area reduction, Related uses the reduction in fully bright viewport candidates, Fisheye uses target-width increase, Smart uses the number of other cards moved and

their total movement, and Overlay uses the ratio of visible edges naturally occluded by cards.

TABLE 5.3: Principal display changes across five behaviour-tree scales

Resource nodes	Adaptive area reduction	Related bright reduction	fully candidate	Fisheye width increase	Smart mean moved cards	Smart mean total movement	Overlay natural occluded-edge ratio
31	33.46%	35.41%		37.04%	2.67	441.15 px	0.1052
61	33.58%	57.91%		54.23%	1.67	284.06 px	0.1869
121	46.07%	68.74%		114.94%	1.67	284.06 px	0.1918
241	57.93%	71.38%		226.33%	3.00	435.11 px	0.0218
364	54.67%	59.10%		300.43%	3.00	435.11 px	0.3677

三项规模敏感功能在五棵真实行为树上的平均效果



FIGURE 5.6: Effects of Adaptive, Related Focus and Fisheye across tree scales

Table 5.3 shows that Adaptive becomes more apparent from 121 nodes upward, because large trees need more area reduction in overview. Related becomes stronger from 61 to 241 nodes, which suggests that unrelated branches become more distracting as the tree becomes more complex. The 364-node tree has a lower Related value than the 241-node tree mainly because the fixed target branch is different and the visible ratio of related to unrelated nodes changes.

Fisheye has the clearest scale trend. This does not mean the on-state card grows without limit. Instead, the off-state target becomes smaller as larger trees are zoomed out, while the on-state target returns to roughly 220 px. Smart removes drag overlap at every scale, but the number of moved cards is not monotonic because it depends on the local branch around the drag point. Overlay is highest on the 364-node tree and lowest on the 241-node tree, showing that it mainly depends on the current viewport and connection path rather than on node count alone.

5.5 RESULTS ACROSS SCREEN SIZES

Table 5.4 summarises the results across three screen sizes. The same behaviour trees and target nodes are used; only the behaviour-tree editor viewport size changes.

TABLE 5.4: Principal display changes under three physical-size profiles

Screen profile	Adaptive area reduction	Related fully bright candidate reduction	Fisheye width increase	Smart mean moved cards	Overlay natural occluded-edge ratio
15.94 in	57.14%	54.01%	258.98%	2.40	0.1682
26.96 in	42.61%	59.61%	104.90%	2.40	0.1735
31.55 in	35.67%	61.90%	75.90%	2.40	0.1825

三项屏幕敏感功能在不同物理尺寸配置下的平均效果

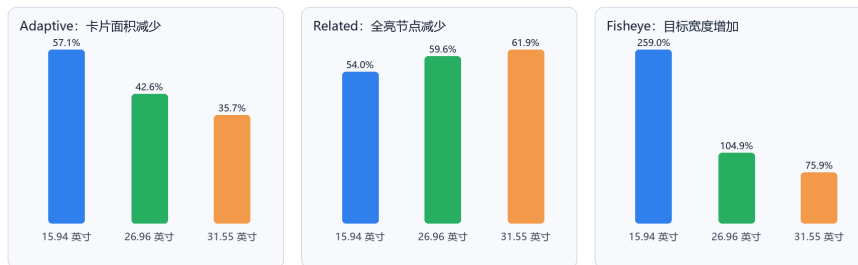


FIGURE 5.7: Effects of Adaptive, Related Focus and Fisheye across screen-size profiles

Table 5.4 shows that a small screen does not make every function more effective. Adaptive and Fisheye have the largest relative effect on the small screen because the overview must shrink the behaviour tree more strongly. Adaptive first reduces global area, and Fisheye can then temporarily enlarge the current target. Their role is therefore to compensate for limited space on a small screen.

Related has a higher reduction on the medium and large screens. A larger screen can show more branches at the same time, and many of those branches are unrelated to the selected node. Related does not move these nodes; it only dims them. This makes it better at using the global space of a large screen. Smart moves the same mean number of cards on all three screens, so this test shows that avoidance is mainly controlled by the drag position. Overlay's natural occlusion ratio rises slightly with screen size because a larger view contains more visible connections and can therefore include more lines passing through cards.

5.6 STRUCTURED DEVELOPER EVALUATION

TABLE 5.5: Structured developer evaluation based on consistent questions

Recommendation order	Display optimization	Most suitable task and environment	Advantage supported by data	Observed cost	Recommendation
1	Adaptive Zoom Detail	Small screens and overviews of 121–364 nodes	Area and fields both fall, with no new hierarchy violation	Fields hidden at low zoom	Most stable general default
2	Smart Drag Reflow	Direct drag editing on all screens	Removes induced overlap in 45/45 cases	Median of 1 other card moved, maximum of 7	Enable by default during editing
3	Related Node Focus	Structural tracing above 121 nodes	Dims every unrelated instance	Makes simultaneous comparison of unrelated branches less convenient	Trigger by default after selection
4	Fisheye Focus	Local inspection of 241–364 nodes on a small screen	Restores targets to at least about 220 px	Increased local overlap and visual hierarchy occlusion	Enable on demand
5	Readable Edge Overlay	A specific connection crossing a card	Keeps the route unchanged and lets part of the hidden segment appear through the background	Local effect and subtle difference	Retain as a situational feature

Table 5.5 does not combine different units into one total score. Instead, it gives a recommendation based on practical use. Adaptive Zoom Detail has the widest coverage and is suitable as the overview foundation. Smart Drag Reflow is the most direct feature when editing node positions. Related Node Focus is useful for checking the structure around the selected node. Fisheye Focus changes the local view most strongly on small screens and large trees, but its side effects are also clearer. Readable Edge Overlay is useful for local line occlusion, so it is closer to a supporting function.

6 DISCUSSION

6.1 PLATFORM CORRECTNESS AS A PREREQUISITE FOR DISPLAY RESEARCH

The tests in Table 5.1 and the completable game scene show that the current tree resources, editor and runtime work together. The display experiment also leaves topology, Decorator ownership, sibling execution order and resource coordinates unchanged. The changes in area, opacity and target size can therefore be attributed to the display features rather than to modified behaviour-tree content.

However, these tests validate only the current node types, scenes and assertions. They do not prove that every future Actor method will be correct, and they do not turn basic behaviour-tree functionality into another research question. The five display optimisations remain the actual objects of discussion.

6.2 INTERPRETATION OF THE DISPLAY OPTIMISATIONS

6.2.1 EFFECTS ACROSS DIFFERENT COMPLEX BEHAVIOUR-TREE SCALES

Small trees already retain comparatively large cards, so the five features are not equally necessary. Table 5.3 shows that Adaptive, Related and fisheye all produce more limited changes on the small tree. Smart still has a direct use because one incorrect drag can cause occlusion, but a small tree normally does not need fisheye to remain active.

As a tree grows, card density and unrelated branches become the main problems. Adaptive first reduces the space occupied by the whole tree in an overview, while Related then makes the selected structure stand out from the other branches. They are not duplicate features: one addresses an overfilled canvas and the other addresses unclear structural relevance. This is why both become more useful on medium and large trees.

Fisheye produces the most apparent local change on the two largest trees. With fisheye disabled, the target has already become very small in the overview; enabling it restores the target to a similar readable width. However, a denser tree makes the enlarged card more likely to cover nearby content. Fisheye is therefore useful for the question “I want to read this local area now”, but it is not a solution for “I want the whole tree to remain tidy”.

The effects of Smart and Overlay depend more on local events than on total node count. Smart handles a collision created by the current drag, while Overlay has an obvi-

ous effect only when a connection passes through a card. Even when a tree is large, the same benefit cannot be assumed for every drag or every connection.

6.2.2 EFFECTS ACROSS PHYSICAL SCREEN SIZES

The screen results show that small and large screens need different forms of support. Cards in a small-screen overview are more likely to become too small, so Adaptive and fisheye have the largest relative effect there. Adaptive first reduces global occupation and fisheye then restores the current local region. They cannot turn a small screen into a large one, but they reduce the effect of limited space when a large tree is inspected.

A large screen already displays more content at once, so further shrinking all cards or enlarging one target has a smaller relative effect. Related Focus has a stronger role here because it preserves the global positions already visible on the large screen and only dims unrelated branches. For structural inspection on a large screen, this is more appropriate than further compression of every node.

The revised fisheye also shows a direct trade-off: local restoration is strongest on the small screen, but so is new occlusion. On a larger screen, the focus is already clearer and the change caused by fisheye is more moderate. A reasonable use is to inspect a target briefly in a small-screen overview and then exit fisheye, rather than using it continuously as the main layout.

Smart and Overlay show no apparent screen difference in the current experiment because the same local operations are used in each profile. These results come from size-profile replays on one GPU. Real viewing distance, pixel density and input devices can still change the experience.

6.2.3 WHICH FEATURES ARE MOST VALUABLE

From practical use in this project, Adaptive is the most suitable feature to keep enabled if only one feature can be selected. It works continuously while the whole tree is browsed and restores complete cards when it is disabled. Its cost is clear: some fields are unavailable while zoomed out, but the user can restore local detail by zooming in or using fisheye.

The answer is different for specific tasks. Smart is the most direct feature during drag editing because it handles the occlusion that has just been created. Related is the most useful for structural inspection because it does not move nodes and retains the related structure of every selected node. The two features do not need to compete: one operates while positions are changed and the other operates while structure is selected and understood.

Fisheye produces the largest relative change in a particular environment, but this does not make it the best general feature. Stronger magnification also creates more local occlusion, so it is more suitable as an on-demand tool. Overlay has the most local scope. It can supplement a connection hidden by a card, but it cannot reduce the number of nodes, fields or candidates.

Overall, Adaptive is suitable as the overview foundation. Smart and Related serve editing and structural inspection respectively, while Fisheye and Overlay are used when a local problem occurs. This conclusion is closer to the real purpose of the five features than adding percentages measured in different units.

6.2.4 WHAT THE EVIDENCE CAN DEMONSTRATE

This study can show the direct changes produced by the five features in the current implementation and can identify the scales and screens on which they are more apparent. Before-and-after figures with the same content help the reader check whether the tables agree with the actual view, while structural signatures confirm that execution semantics have not changed.

However, these data cannot show how much faster an ordinary developer would complete a task, and reduced card area cannot be described directly as reduced cognitive load. The structured developer evaluation only turns the present data into design recommendations for this project. If participants can be recruited later, the same trees and tasks can be retained while completion time, incorrect selections, zoom and pan counts, and subjective ratings are recorded.

7 CONCLUSION

This project completed an editable, saveable, executable and debuggable behaviour-tree plugin for Godot 4.6 and used it to study the display of large behaviour trees. The current Display menu contains Smart Drag Reflow, Adaptive Zoom Detail, Readable Edge Overlay, Related Node Focus and Fisheye Focus. The experiment uses five genuine game trees and three screen-size profiles taken from real devices.

Tables 5.2 to 5.4 show that all five features produce the type of change they target without changing tree structure or execution order. Smart is suitable for occlusion caused by dragging, Adaptive for continuous inspection of the complete tree, and Related for inspecting the structural relationships of selected nodes. The revised fisheye provides stronger local magnification on small screens and large trees, but it also creates more occlusion. Overlay only addresses a particular connection hidden by a card.

The research question can therefore be answered with a conditional yes: these features improve the measurable display or editing state that each targets, but no feature is best at every scale, screen and task. Overall, Adaptive is the most suitable general feature. Smart and Related are appropriate for editing and structural inspection respectively, while Fisheye and Overlay are more suitable for on-demand use.

The principal value of this project is not to prove that behaviour trees are superior to other AI architectures. It places several large-graph display methods in a genuine executable behaviour-tree editor and allows them to be enabled, disabled and compared. Future work can ask external game developers to perform the same tasks and then check whether these geometric and display changes actually reduce operation time and errors.

REFERENCES

- Archambault, D. and Purchase, H. C. (2013) ‘The “Map” in the Mental Map: Experimental Results in Dynamic Graph Drawing’, *International Journal of Human-Computer Studies*, 71(11), pp. 1044–1055. Available at: <https://doi.org/10.1016/j.ijhcs.2013.08.004>.
- Bederson, B. B., Hollan, J. D., Perlin, K., Meyer, J., Bacon, D. and Furnas, G. W. (1996) ‘Pad++: A Zoomable Graphical Sketchpad for Exploring Alternate Interface Physics’, *Journal of Visual Languages & Computing*, 7(1), pp. 3–32. Available at: <https://doi.org/10.1006/jvlc.1996.0002>.
- Büring, T., Gerken, J. and Reiterer, H. (2006) ‘User Interaction with Scatterplots on Small Screens: A Comparative Evaluation of Geometric-Semantic Zoom and Fisheye Distortion’, *IEEE Transactions on Visualization and Computer Graphics*, 12(5), pp. 829–836. Available at: <https://doi.org/10.1109/TVCG.2006.187>.
- Cockburn, A., Karlson, A. and Bederson, B. B. (2009) ‘A Review of Overview+Detail, Zooming, and Focus+Context Interfaces’, *ACM Computing Surveys*, 41(1), Article 2, pp. 1–31. Available at: <https://doi.org/10.1145/1456650.1456652>.
- Colledanchise, M. and Ögren, P. (2017) ‘How Behavior Trees Modularize Hybrid Control Systems and Generalize Sequential Behavior Compositions, the Subsumption Architecture, and Decision Trees’, *IEEE Transactions on Robotics*, 33(2), pp. 372–389. Available at: <https://doi.org/10.1109/TRO.2016.2633567>.
- Colledanchise, M. and Ögren, P. (2018) *Behavior Trees in Robotics and AI: An Introduction*. Boca Raton, FL: CRC Press. Available at: <https://doi.org/10.1201/9780429489105>.
- Di Bartolomeo, S., Crnovrsanin, T., Saffo, D., Puerta, E., Wilson, C. and Dunne, C. (2024) ‘Evaluating Graph Layout Algorithms: A Systematic Review of Methods and Best Practices’, *Computer Graphics Forum*, 43(6), e15073. Available at: <https://doi.org/10.1111/cgf.15073>.
- Dwyer, T., Marriott, K. and Stuckey, P. J. (2006) ‘Fast Node Overlap Removal’, in Healy, P. and Nikolov, N. S. (eds.) *Graph Drawing 2005*, LNCS 3843, pp. 153–164. Available at: https://doi.org/10.1007/11618058_15.
- Dwyer, T., Marriott, K. and Wybrow, M. (2009) ‘Topology Preserving Constrained Graph Layout’, in Tollis, I. G. and Patrignani, M. (eds.) *Graph Drawing 2008*, LNCS 5417, pp. 230–241. Available at: https://doi.org/10.1007/978-3-642-00219-9_22.
- Flórez-Puga, G., Gómez-Martín, M. A., Gómez-Martín, P. P., Díaz-Agudo, B. and González-Calero, P. A. (2009) ‘Query-Enabled Behavior Trees’, *IEEE Transactions*

on *Computational Intelligence and AI in Games*, 1(4), pp. 298–308. Available at: <https://doi.org/10.1109/TCIAIG.2009.2036369>.

Furnas, G. W. (1986) ‘Generalized Fisheye Views’, in *Proceedings of CHI ’86*, pp. 16–23. Available at: <https://doi.org/10.1145/22627.22342>.

Gansner, E. R. and Hu, Y. (2010) ‘Efficient, Proximity-Preserving Node Overlap Removal’, *Journal of Graph Algorithms and Applications*, 14(1), pp. 53–74. Available at: <https://doi.org/10.7155/jgaa.00198>.

Ghoniem, M., Fekete, J.-D. and Castagliola, P. (2005) ‘On the Readability of Graphs Using Node-Link and Matrix-Based Representations: A Controlled Experiment and Statistical Analysis’, *Information Visualization*, 4(2), pp. 114–135. Available at: <https://doi.org/10.1057/palgrave.ivs.9500092>.

Helmke, S., Doğan, K., Scheffler, R. and Wrobel, G. (2024) ‘Domain-Specific Rules Override Aesthetic Graph Drawing Criteria: An Exploration of User-Generated Diagrams’, in *Diagrammatic Representation and Inference: Diagrams 2024*, LNCS 14981, pp. 44–60. Available at: https://doi.org/10.1007/978-3-031-71291-3_4.

Holten, D. (2006) ‘Hierarchical Edge Bundles: Visualization of Adjacency Relations in Hierarchical Data’, *IEEE Transactions on Visualization and Computer Graphics*, 12(5), pp. 741–748. Available at: <https://doi.org/10.1109/TVCG.2006.147>.

Iovino, M., Scutkins, E., Styrd, J., Ögren, P. and Smith, C. (2022) ‘A Survey of Behavior Trees in Robotics and AI’, *Robotics and Autonomous Systems*, 154, 104096. Available at: <https://doi.org/10.1016/j.robot.2022.104096>.

Jakobsen, M. R. and Hornbæk, K. (2013) ‘Interactive Visualizations on Large and Small Displays: The Interrelation of Display Size, Information Space, and Scale’, *IEEE Transactions on Visualization and Computer Graphics*, 19(12), pp. 2336–2345. Available at: <https://doi.org/10.1109/TVCG.2013.170>.

Lim, C.-U., Baumgarten, R. and Colton, S. (2010) ‘Evolving Behaviour Trees for the Commercial Game DEFCON’, in *Applications of Evolutionary Computation: EvoApplications 2010*, LNCS 6024, pp. 100–110. Available at: https://doi.org/10.1007/978-3-642-12239-2_11.

Marzinotto, A., Colledanchise, M., Smith, C. and Ögren, P. (2014) ‘Towards a Unified Behavior Trees Framework for Robot Control’, in *2014 IEEE International Conference on Robotics and Automation*, pp. 5420–5427. Available at: <https://doi.org/10.1109/ICRA.2014.6907656>.

McGee, F. and Dingliana, J. (2012) ‘An Empirical Study on the Impact of Edge Bundling on User Comprehension of Graphs’, in *Proceedings of the International Working Conference on Advanced Visual Interfaces*, pp. 620–627. Available at: <https://doi.org/10.1145/2254556.2254670>.

Misue, K., Eades, P., Lai, W. and Sugiyama, K. (1995) ‘Layout Adjustment and the Mental Map’, *Journal of Visual Languages & Computing*, 6(2), pp. 183–210. Available at: <https://doi.org/10.1006/jvlc.1995.1010>.

- Plaisant, C., Grosjean, J. and Bederson, B. B. (2002) ‘SpaceTree: Supporting Exploration in Large Node Link Tree, Design Evolution and Empirical Evaluation’, in *IEEE Symposium on Information Visualization*, pp. 57–64. Available at: <https://doi.org/10.1109/INFVIS.2002.1173148>.
- Purchase, H. C. (1997) ‘Which Aesthetic Has the Greatest Effect on Human Understanding?’, in Di Battista, G. (ed.) *Graph Drawing, GD 1997*, LNCS 1353, pp. 248–261. Available at: https://doi.org/10.1007/3-540-63938-1_67.
- Purchase, H. C., Carrington, D. and Allder, J.-A. (2002) ‘Empirical Evaluation of Aesthetics-Based Graph Layout’, *Empirical Software Engineering*, 7(3), pp. 233–255. Available at: <https://doi.org/10.1023/A:1016344215610>.
- Reingold, E. M. and Tilford, J. S. (1981) ‘Tidier Drawings of Trees’, *IEEE Transactions on Software Engineering*, SE-7(2), pp. 223–228. Available at: <https://doi.org/10.1109/TSE.1981.234519>.
- Sarkar, M. and Brown, M. H. (1994) ‘Graphical Fisheye Views’, *Communications of the ACM*, 37(12), pp. 73–84. Available at: <https://doi.org/10.1145/198366.198384>.
- Sugiyama, K., Tagawa, S. and Toda, M. (1981) ‘Methods for Visual Understanding of Hierarchical System Structures’, *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2), pp. 109–125. Available at: <https://doi.org/10.1109/TSMC.1981.4308636>.
- Summers, K. L., Goldsmith, T. E., Kubica, S. and Caudell, T. P. (2003) ‘An Experimental Evaluation of Continuous Semantic Zooming in Program Visualization’, in *IEEE Symposium on Information Visualization*, pp. 155–162. Available at: <https://doi.org/10.1109/INFVIS.2003.1249021>.
- van Ham, F. and Perer, A. (2009) ‘Search, Show Context, Expand on Demand: Supporting Large Graph Exploration with Degree-of-Interest’, *IEEE Transactions on Visualization and Computer Graphics*, 15(6), pp. 953–960. Available at: <https://doi.org/10.1109/TVCG.2009.108>.
- Ware, C. and Bobrow, R. (2005) ‘Supporting Visual Queries on Medium-Sized Node-Link Diagrams’, *Information Visualization*, 4(1), pp. 49–58. Available at: <https://doi.org/10.1057/palgrave.ivs.9500090>.
- Ware, C., Purchase, H., Colpoys, L. and McGill, M. (2002) ‘Cognitive Measurements of Graph Aesthetics’, *Information Visualization*, 1(2), pp. 103–110. Available at: <https://doi.org/10.1057/palgrave.ivs.9500013>.
- Wang, Y., Wang, Y., Zhang, H., Sun, Y., Fu, C.-W., Sedlmair, M., Chen, B. and Deussen, O. (2019) ‘Structure-Aware Fisheye Views for Efficient Large Graph Exploration’, *IEEE Transactions on Visualization and Computer Graphics*, 25(1), pp. 566–575. Available at: <https://doi.org/10.1109/TVCG.2018.2864911>.
- Wong, N., Carpendale, S. and Greenberg, S. (2003) ‘EdgeLens: An Interactive Method for Managing Edge Congestion in Graphs’, in *IEEE Symposium on Information Visualization*, pp. 51–58. Available at: <https://doi.org/10.1109/INFVIS.2003.1249008>.
- Zhou, Z., Jin, J., Phadnis, V., Yuan, X., Jiang, J., Qian, X., Wright, K., Sherwood, M., Mayes, J., Zhou, J., Huang, Y., Xu, Z., Zhang, Y., Lee, J., Olwal, A., Kim,

D., Iyengar, R., Li, N. and Du, R. (2025) ‘InstructPipe: Generating Visual Blocks Pipelines with Human Instructions and LLMs’, in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, Article 877, pp. 1–22. Available at: <https://doi.org/10.1145/3706598.3713905>.